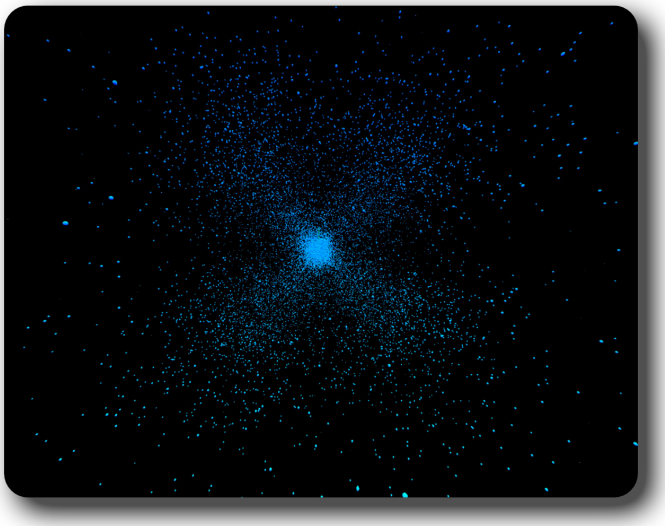
Hochschule Konstanz
Technik, Wirtschaft und Gestaltung

Optimierung der Simulation des n-Body Problems

Vergleich und Analyse verschiedener Parallelisierungsmethoden im Rahmen des Moduls "Parallel Computing"

Florian Dritter und Tom Berger

Konstanz, 6. Februar 2026



Hochschule:	HTWG Konstanz
Fakultät:	Informatik
Studiengang:	Angewandte Informatik
Modul:	Parallel Computing
Dozent:	Marco Fehrenbach

Inhaltsverzeichnis

Inhaltsverzeichnis ii

1 Einleitung 1
1.1 Einführung in das n-Body Problem . . . 1
1.2 Vorgehensweise bei der Performance-Messung . . . 2

2 Analyse der Parallelität 3

3 Optimierung der n-Body Simulation 4
3.1 Sequenzielle Implementierung . . . 4
3.2 OpenMP Parallelisierung . . . 4
3.2.1 Structure of Arrays . . . 4
3.2.2 Kontrollfluss und SIMD Optimierung . . . 6
3.2.3 Algorithmus ohne Verschachtelte Schleifen . . . 7
3.3 MPI Parallelisierung . . . 9
3.3.1 Mehrere Ranks auf einer Node . . . 10
3.3.2 Load-Balancing . . . 10
3.3.3 MPI in Windows . . . 11
3.3.4 MPI in Docker . . . 11
3.3.5 Auswertung . . . 12
3.4 OpenCL Parallelisierung . . . 13
3.4.1 OpenCL-Umsetzung bisheriger Optimierungen . . . 13
3.4.2 Shared Memory . . . 13
3.5 Abweichungen im Ergebnis zwischen Algorithmen . . . 15
3.6 Zusammenfassung . . . 16

4 Ausblick 17
4.1 MPI und OpenCL . . . 17
4.2 Barnes-Hut Algorithmus . . . 17

Literaturverzeichnis 18

Abbildungsverzeichnis 19

Tabellenverzeichnis 20

1 Einleitung

1.1 Einführung in das n-Body Problem

Das n-Body Problem ist ein klassisches Problem der Himmelsmechanik und beschreibt die Bewegung von n Himmelskörpern, die sich gegenseitig aufgrund ihrer Gravitationskräfte beeinflussen. Während das Zwei-Körper-Problem eine exakte Lösung hat, wird das n-Body Problem für n größer als zwei zunehmend komplexer und erfordert numerische Methoden zur Berechnung der Bewegungen der Körper über die Zeit [3, Kap. 1]. Da sich dieses Projekt jedoch nicht auf die Physik, sondern ausschließlich auf die Softwareentwicklung einer Simulation des n-Body Problems konzentriert, soll der Folgende Abschnitt nicht als vollständige Erklärung sondern vielmehr als grundlegende Einführung in die physikalischen Hintergründe dienen.

Die einschlägige Methode zur Berechnung der Gravitationskräfte zwischen den Körpern lässt sich für einen beliebigen Körper i durch das Gravitationsgesetz von Newton ausdrücken [6, Kap. 11.1]:

$$\vec{F_i} = G \sum_{\substack{j=1 \\ j \neq i}}^{n} \frac{m_i m_j}{r_{ij}^2} \hat{r}_{ij} \tag{1}$$

Hierbei ist G die Gravitationskonstante, m_i und m_j die Massen der Körper i und j, r_{ij} der Abstand zwischen den Körpern, und $\hat{r}_{ij}$ der Einheitsvektor von Körper i zu Körper j. Die Gesamtbeschleunigung eines Körpers ergibt sich aus der Summe der Gravitationskräfte aller anderen Körper, dividiert durch seine Masse:

$$\vec{a_i} = \frac{\vec{F_i}}{m_i} \tag{2}$$

Woraus für sich die neue Geschwindigkeit für den nächsten Zeitschritt Δt ableiten lässt:

$$\begin{aligned} \vec{v_i}(t + \Delta t) &= \vec{v_i}(t) + \vec{a_i} \Delta t \\ &= \vec{v_i}(t) + \frac{\vec{F_i}}{m_i} \Delta t \\ &= \boxed{\vec{v_i}(t) + G \sum_{\substack{j=1, \\ j \neq i}}^{n} \frac{m_j}{r_{ij}^2} \hat{r}_{ij} \Delta t} \end{aligned} \tag{3}$$

Und schließlich die neue Position, wobei hier das semi-implizite Euler-Verfahren (auch symplektischer Euler genannt) angewendet wird, um die Stabilität der Bahnen zu gewährleisten [4, Abschn. 17.4]:

$$\vec{p_i}(t + \Delta t) = \vec{p_i}(t) + \vec{v_i}(t + \Delta t) \Delta t \tag{4}$$

Da die Simulation nur mit diskreten Zeitschritten arbeiten kann, werden die Positionen und Geschwindigkeiten der Körper in jedem Zeitschritt aktualisiert. Der grundlegende Ablauf eines Simulationsschritts umfasst dabei folgende Schritte:

1. Berechnung der Gravitationskräfte zwischen allen Körpern in ihren aktuellen Positionen.
2. Bestimmung der Beschleunigungen basierend auf den Kräften.
3. Aktualisierung der Geschwindigkeiten.

4. Aktualisierung der Positionen basierend auf den neuen Geschwindigkeiten.

Dieser Prozess wird iterativ wiederholt, um die Bewegung der Körper über die Zeit zu simulieren. Die Genauigkeit der Simulation hängt dabei stark von der Wahl des Zeitschritts ab.

1.2 Vorgehensweise bei der Performance-Messung

Die nachfolgenden Performance-Messungen wurden - sofern nicht ausdrücklich anders angegeben - auf einem MacBook Air mit Apple M2 (2022) Chip und 16 GB Arbeitsspeicher durchgeführt.

Dabei wurden für jede untersuchte Kombination aus Algorithmus und Körper-Anzahl wenn nicht anders angegeben 10 unabhängige Zeitmessungen in Millisekunden über 750 Zeitschritte durchgeführt und deren Mittelwert berechnet. Bei Balkendiagrammen wird zusätzlich die Standardabweichung als Fehlerbalken dargestellt, wobei diese bei den meisten Messungen so gering ist, dass sie kaum sichtbar ist.

2 Analyse der Parallelität

Die Berechnung einer n-Body-Simulation lässt sich in mehrere Abstraktionsebenen unterteilen.

Auf äußerster Ebene ist die vollständige Berechnung eines Zeitschritts in Form einer Map-Operation. Pro Zeitschritt werden alle n-Körper auf n-Körper mit neuen Eigenschaften abgebildet. Da jeder Körper abhängig von seiner Position eine Kraft auf jeden anderen Körper im System ausübt, muss die Berechnung des vorherigen Zeitschrittes immer vollständig abgeschlossen sein, damit die Berechnung eines neuen Zeitschritts beginnen kann. Daher kann diese Ebene nicht parallelisiert werden.

Die Berechnung der neuen Position eines Körpers hingegen basiert nur auf den alten Positionen aller anderen Körper. Das lässt sich perfekt parallelisieren, da es eine Gather-Operation ist, die nur von den Umständen des zuletzt berechneten Zeitschrittes abhängt. Alle diese Berechnungen sind unabhängig voneinander und unterscheiden sich nur in ihren Daten, sind also Daten-Parallel.

Innerhalb dieser Berechnung wird eine Reduktionsoperation durchgeführt, bei der die wirkenden Kräfte, und daraus resultierenden Beschleunigungen, für den jeweiligen Körper zusammengerechnet werden. Theoretisch lässt sich dieser Vorgang also auch zumindest teilweise parallelisieren.

Auch wenn die Positionsberechnung durch die innere Reduktion zwar nicht günstig ist, ist sie doch im Hinblick auf die Gesamtberechnung der Simulation eine sehr fein-granulare Operation. Da diese Berechnung auch Daten-Parallel ist, eignet sie sich hervorragend für die Ausführung auf der GPU.

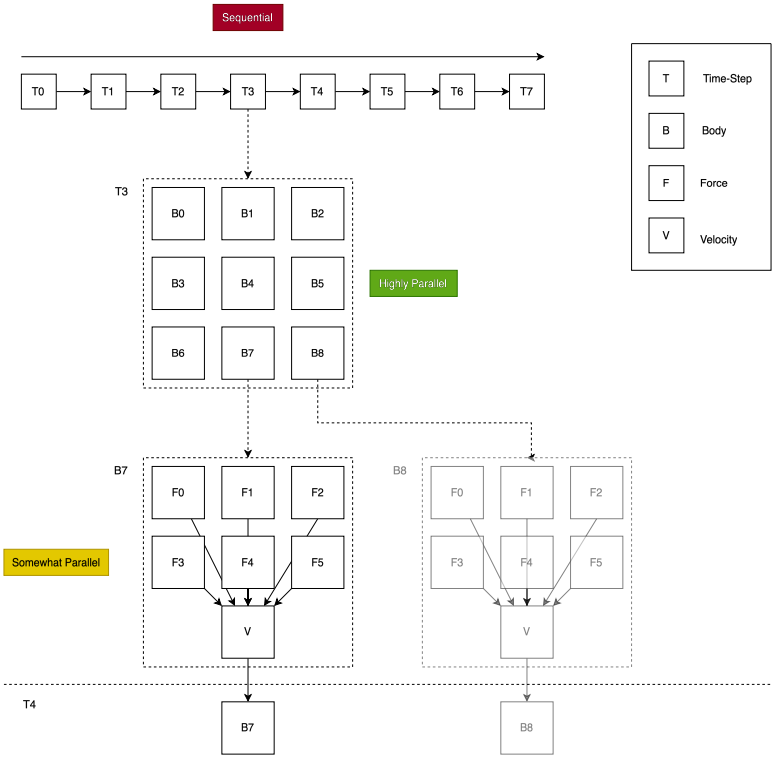


Abbildung 1: n-Body-Simulation Paralellisierbarkeit

3 Optimierung der n-Body Simulation

Um die Geschwindigkeit und Effizienz der n-Body Simulation zu verbessern, wurden verschiedene Optimierungsstrategien implementiert. Diese umfassen algorithmische Verbesserungen, Datenstrukturoptimierungen und die Nutzung von Parallelisierungstechniken. In den folgenden Abschnitten werden die versuchten Optimierungen detailliert beschrieben und deren Auswirkungen auf die Performance der Simulation analysiert.

3.1 Sequenzielle Implementierung

Die erste Implementierung der n-Body Simulation wurde als sequenzieller Algorithmus realisiert. Hierbei wurden die Gravitationskräfte zwischen allen Körpern in einer einfachen Schleife berechnet, was zu einer Zeitkomplexität von $O(n^2)$ führte. Diese Implementierung diente als Basis für weitere Optimierungen und ermöglichte es, die Korrektheit der Simulation zu überprüfen. Da sie sich sehr stark an den physikalischen Grundlagen orientiert, werden im Folgenden keine weiteren Details dazu beschrieben.

3.2 OpenMP Parallelisierung

Um die Performance der n-Body Simulation zu verbessern, wurde die OpenMP-Bibliothek zur Parallelisierung des Algorithmus eingesetzt. OpenMP ermöglicht es, Schleifen und Abschnitte des Codes auf mehrere Threads zu verteilen, wodurch die Berechnungen auf Mehrkernprozessoren beschleunigt werden können.

In der n-Body Simulation wurde die Berechnung der Gravitationskräfte zwischen den Körpern parallelisiert, indem die äußere Schleife, die über alle Körper iteriert, mit einer OpenMP-Direktive versehen wurde.
OpenMP erstellt dadurch eine Reihe von Threads (normalerweise einen pro Hardware-Kern) und teilt die Iterationen der Schleife unter ihnen auf, sodass die CPU mehrere Teile der Schleife gleichzeitig berechnen kann.
Da jede Iteration den gleichen Rechenaufwand erfordert, können wir *static* Scheduling verwenden, wodurch jedem Thread eine feste, durchgehende und gleich große Menge an Iterationen zugewiesen wird, und die Speicherlokalität verbessert wird.

Dies führte, wie in Abbildung 2 zu sehen, zu einer signifikanten Reduktion der Rechenzeit, insbesondere bei einer großen Anzahl von Körpern.

3.2.1 Structure of Arrays

Diese erste Implementierung der Simulation basierte auf der intuitiv naheliegendsten Datenstruktur: Körper wurden als Structs mit Feldern wie Position, Masse und Geschwindigkeit in einem Array, also einer klassichen *Array of Structures (AoS)* Struktur, gespeichert.
Das ist konzeptionell sauber und erleichtert die Implementierung, ist aber performance-technisch für moderne CPUs oft ein Nachteil.

Der enstehende Engpass ist in der inneren Schleife, in der die Kräfte auf einen Körper zusammenaddiert werden. Für diese Berechnung benötigt man nur die Position und die Masse, doch aufgrund der AoS Struktur werden die Körper vollständig mitsamt der Geschwindigkeit in den Speicher geladen.

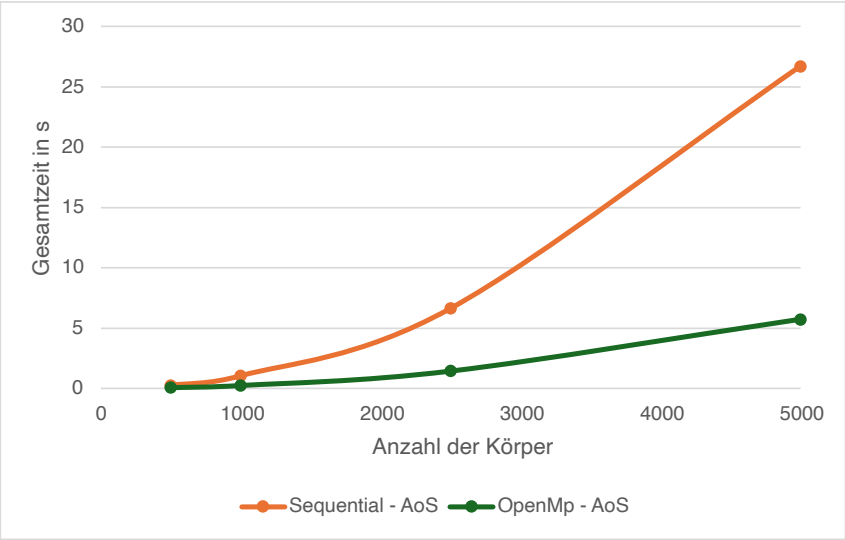


Abbildung 2: Geschwindigkeit Sequenziell vs. OpenMP

Das führt zu sehr ineffizienter Speichernutzung. Die Geschwindigkeitswerte verschwenden Platz im Cache und in der Memory-Bandbreite und die automatische SIMD-Vektorisierung der CPU wird dadurch erschwert, dass Werte wie bspw. alle x-Positionswerte nicht zusammenhängend im Memory liegen.[5]

Dieses Problem kann gelöst werden, indem man die Datenstruktur „umdreht", also statt eines Arrays von einer Struktur mit mehreren Feldern mehrere Arrays mit den Werten einzelner Felder verwendet.

```
struct Body {
    float mass;
    glm::vec3 position;
    glm::vec3 velocity;
};

std::vector<Body> bodies;
```

Abbildung 3: Bodies als AoS

```
struct BodiesSoA {
    std::vector<glm::vec3> positions;
    std::vector<glm::vec3> velocities;

    std::vector<float> masses;
};
```

Abbildung 4: Bodies als SoA

Diese erste Implementierung der Structure of Arrays stellte sich allerdings als unzureichend heraus. Zwar wurden nun nicht mehr unbrauchbare Werte in den Speicher geladen, eine wirklich messbare Performance-Verbesserung wurde allerdings nicht erreicht - auch hier bremste die Vermischung von x-, y- und z-Koordinaten die Vektorisierung weiterhin stark aus.

In einer zweiten, konsequenteren Iteration wurden die letzten *vec3* Structs in reine *float* arrays aufgeteilt, sodass jeder Koordinatentyp zusammenhängend in den Speicher geladen werden konnte.

```
struct BodiesSoA {
    std::vector<float> posX;
    std::vector<float> posY;
    std::vector<float> posZ;

    std::vector<float> velX;
    std::vector<float> velY;
    std::vector<float> velZ;

    std::vector<float> masses;
};
```

Abbildung 5: Bodies als verbesserte SoA

Mit dieser Optimierung konnte dann ein deutlicher Geschwindigkeits-Zuwachs gemessen werden (vgl. Abbildung 9, 10).

Zur besseren Vergleichbarkeit wurde auch nachträglich eine SoA Implementierung für die Sequenzielle Lösung erstellt, die auch bereits eine Verbesserung zur AoS Version aufwies (vgl. Abbildung 6.

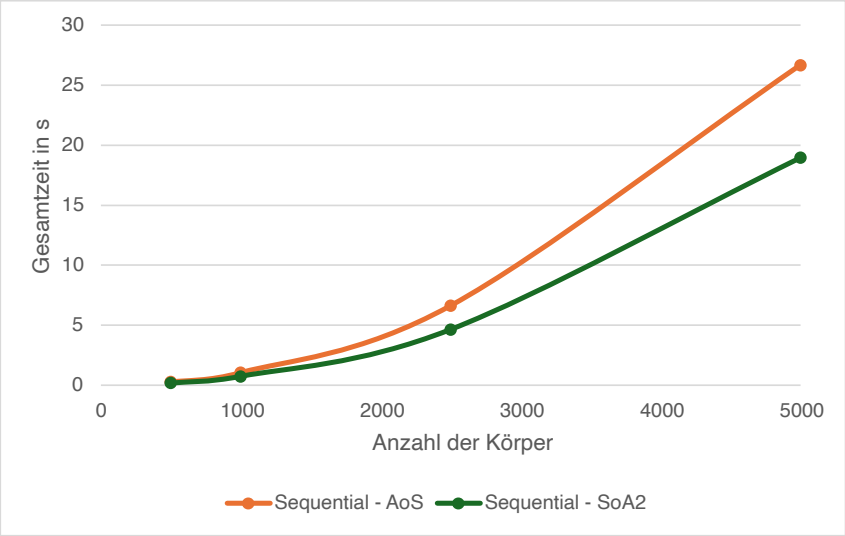


Abbildung 6: Sequenziell AoS vs SoA

3.2.2 Kontrollfluss und SIMD Optimierung

Zwar wurde die Vektorisierung durch die SoA Struktur bereits deutlich verbessert, doch die Schleife enthielt noch eine weitere Bremse. Um zu verhindern, dass Körper auch eine Kraft auf sich selbst auswirken, muss die Schleife überprüfen, ob der aktuelle Index und der des Ziel-Körpers gleich sind.
Dieses If-Statement erzeugt Branching im wichtigsten und am häufigsten ausgeführten Code-Abschnitt, obwohl nur ein einziger Fall Mal pro Berechnung eines Körpers abgedeckt wird. Trotzdem behindert es das Laden von CPU-Instruktionen und die SIMD-Vektorisierung. [5]

```
for (int j = 0; j < bodies.size(); j++) {
    if (j ≠ targetBodyIndex) {
        // Berechnung...
    }
}
```

Abbildung 7: If-Statement innere Schleife

Umgehen kann man diese Verzweigung durch Aufteilung der Berechnung in zwei Bereiche um den aktuellen Index herum, die von unterschiedlichen Schleifen berechnet werden.

```
#pragma omp simd
for (int j = 0; j < targetBodyIndex; ++j) {
    accumulateForce(j);
}

#pragma omp simd
for (int j = targetBodyIndex + 1; j < bodies.size(); ++j) {
    accumulateForce(j);
}
```

Abbildung 8: Innere Schleife Split-Loop Lösung

Ergebnis dieser Optimierung war eine erneute, deutlich messbare Geschwindigkeitsverbesserung (vgl. Abbildung 9, 10).

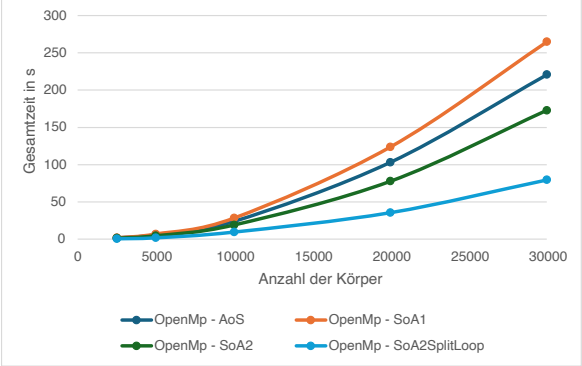


Abbildung 9: OMP Vergleich

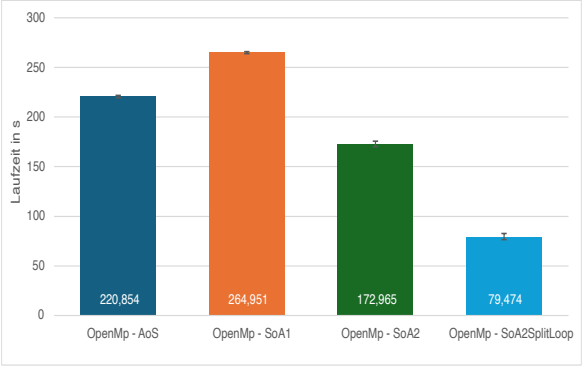


Abbildung 10: OMP Vergleich - 30k

3.2.3 Algorithmus ohne Verschachtelte Schleifen

Eine weitere Optimierungsidee erwuchs aus der Tatsache, dass zwei Körper sich gegenseitig mit der gleichen Kraft beeinflussen. Wenn man es also hinbekommt, die Berechnung so zu gestalten, dass die Kraft zwischen zwei Körpern nur einmal berechnet wird, könnte man die Anzahl der Berechnungen von $O(n^2)$ auf $O(n(n-1)/2)$ reduzieren, was bei großen n einen

erheblichen Performance-Gewinn bedeuten würde.

Um das zu erreichen muss man die Berechnung in zwei Schritte aufteilen: Zuerst wird die Kraft zwischen allen Körper-Paaren berechnet und in einer temporären Datenstruktur gespeichert. Danach werden diese Kräfte dann auf die jeweiligen Körper angewendet, um deren neue Positionen zu berechnen. Um später nicht mehr die Beschleunigungen in einem weiteren extra Schleifendurchlauf berechnen zu müssen, werden die Kräfte direkt in Beschleunigungen umgerechnet und gespeichert.

In der Praxis hat diese Lösung allerdings zwei große Probleme. Zum einen benötigt die temporäre Datenstruktur sehr viel Speicherplatz, da für jedes Körper-Paar eine Beschleunigung gespeichert werden muss. Bei n Körpern sind das $n(n-1)/2$ Einträge, was bei großen n schnell unpraktikabel wird.

Zum anderen ist die Zugriffsstruktur auf diese Datenstruktur sehr unregelmäßig, was zu Cache-Misses und ineffizienter Speichernutzung führt. Da die Körper in der temporären Struktur nicht in der Reihenfolge ihrer Indizes gespeichert sind, sondern nach Paaren, ist es schwierig, die Daten effizient zu laden und zu verarbeiten. Außerdem ist die Berechnung der Indizes für den Zugriff auf die temporäre Struktur komplex und führt zu zusätzlichem Overhead.

Aufgrund dieser Probleme konnte die Single-Loop Implementierung in der Praxis keine Performance-Verbesserung erzielen, sondern war sogar deutlich langsamer als die vorherige Array-of-Structures geschweigedenn die Split-Loop Version (vgl. Abbildung 9, 10).

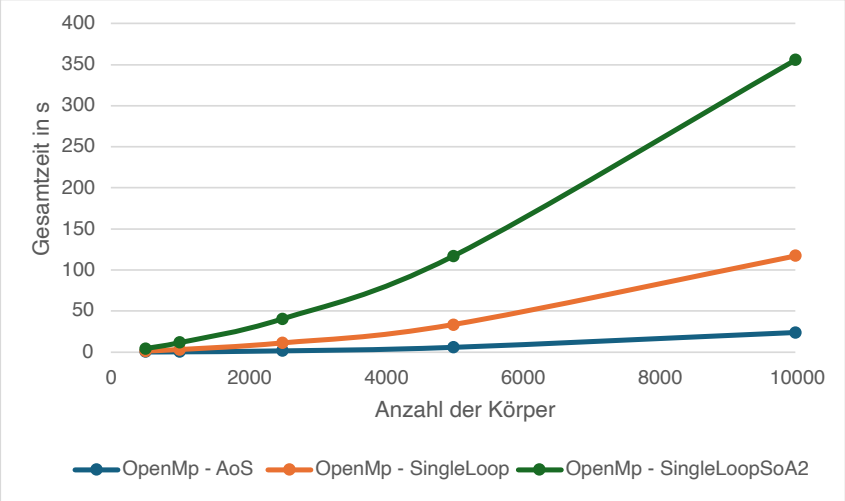


Abbildung 11: OMP Single-Loop im Vergleich zu OMP AoS

Desweiteren fiel bei Tests der beiden Single-Loop Versionen auf, dass sich die Performance für diesen Algorithmus massiv zwischen dem MacBook Air mit Apple M2 und der Workstation mit Ryzen 9 7900 unterschieden. Während die Array-of-Structures Single-Loop Version auf dem MacBook Air sogar schneller war, überzeugte auf der Workstation die verbesserte SoA Version deutlich.
Wirft man jedoch einen Blick auf die genauen Werte, so erkennt man, dass auf dem MacBook die Single-Loop Versionen durchgehend schneller sind, als auf der eigentlich leistungsstärkeren Workstation - das lässt sich nicht zuletzt auch an der zum Vergleich herangezogenen AoS OMP Version erkennen, die auf der Workstation deutlich schneller ist als auf dem MacBook.

Das MacBook Air scheint also die Paralellisierungsmöglichkeiten der CPU deutlich besser auszunutzen, als die Workstation, was auch bei anderen Tests während dieses Projekts der Fall war.

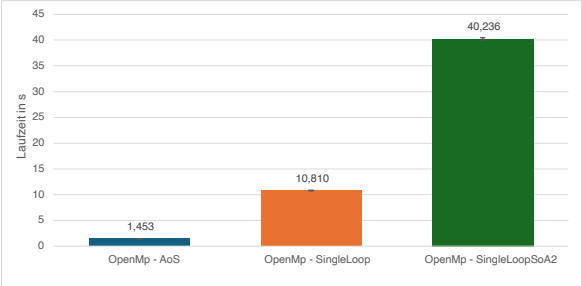


Abbildung 12: Single-Loop Version auf dem MacBook Air (2500 Körper)

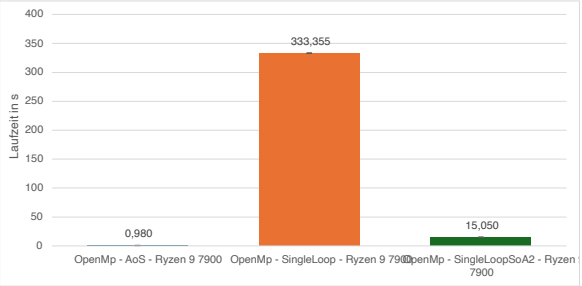


Abbildung 13: Single-Loop Version auf der Workstation (2500 Körper)

3.3 MPI Parallelisierung

Zwar bieten CPUs mit Threads die Möglichkeit mehrere Instruktionen zur selben Zeit, also echt parallel, auszuführen, doch ist die Anzahl dieser parallelen Instruktionen durch die Anzahl der CPU-Kerne hart limitiert.
Die meisten Consumer-CPUs verfügen über 8 bis 16 Kerne, und während Datacenter-CPUs mehrere hundert bis sogar tausend Kerne erreichen [1][2], ist das Potenzial physikalisch doch schnell ausgeschöpft.

Abhilfe schafft hier der MPI-Standard. Mit Hilfe von MPI können parallele Berechnungen auf mehrere Prozesse (Ranks), die auf eigenen Maschinen (Nodes) laufen, aufgeteilt werden. Dadurch stehen - theoretisch - unendlich viele parallele Recheneinheiten zur Verfügung.

In der Praxis bremst vor allem die Kommunikation zwischen den Prozessen die Leistungsfähigkeit von MPI aus, wodurch sich Workloads mit minimalen Abhängigkeiten voneinander am Besten für Parallelisierung durch MPI eignen.
Trotzdem kann MPI bereits bei wenigen verbundenen Nodes erhebliche Geschwindigkeits-Verbesserungen mit sich bringen.

Zu Testzwecken zur Verfügung standen die in Tabelle 1 gelisteten Nodes.

Device	CPU	RAM	OS
Workstation	Ryzen 9 7900	64 GB	Windows / Linux
ThinkPad	i5-8250U	32 GB	Linux
IdeaPad	Ryzen 7 8845HS	32 GB	Windows
MacBook Air	Apple M2	16 GB	macOS
HP	i5-6500	16 GB	Linux

Tabelle 1: Verfügbare Nodes

Um den Algorithmus für die Ausführung auf mehreren Nodes vorzubereiten, wurden die Berechnungen so umgestaltet, dass sie in unabhängige Blöcke aufgeteilt werden können, die

von verschiedenen Ranks berechnet werden können. Dafür müssen zwar alle Ranks in jedem Zeitschritt die aktuellen Positionen aller Körper kennen, um die Kräfte korrekt berechnen zu können, doch die Berechnung der neuen Positionen kann dann von jedem Rank für einen Teil der Körper durchgeführt werden. Danach müssen die Ranks ihre Ergebnisse synchronisieren, damit alle Ranks die neuen Positionen aller Körper für den nächsten Zeitschritt haben.

3.3.1 Mehrere Ranks auf einer Node

Wie zuvor erwähnt liegt die Stärke von MPI in der Zusammenarbeit von mehreren Nodes. Man kann allerdings auch mehrere Ranks auf einer einzigen Node laufen lassen.
Im direkten Vergleich mit OpenMP offenbart sich darin eindrücklich der Overhead, der gegenüber Threads durch die Kommunikation und Context Switching zwischen den Prozessen entsteht.

Zu diesem Zweck wurde die Laufzeit einer Sequenzielle AoS Implementation in MPI mit der einer AoS Implementation von OpenMP verglichen.

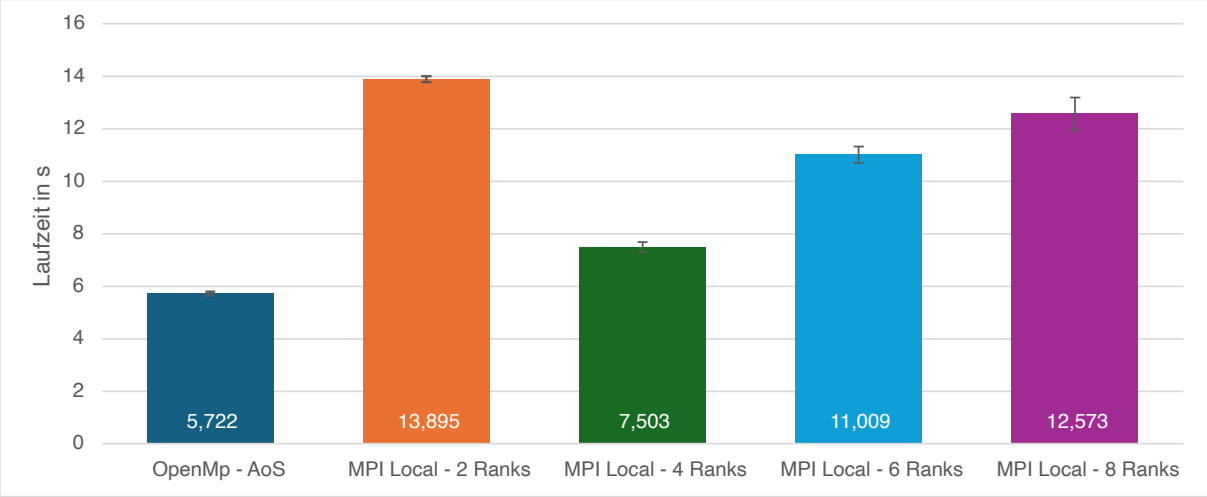


Abbildung 14: OpenMP AoS vs. MPI auf einer Node (5k Körper)

Auffällig ist hier, dass - obwohl die Testmaschine 8 Kerne hat - MPI am schnellsten ist, wenn vier Instanzen erzeugt werden. Bei höheren Anzahlen überwiegt der Overhead in der Kommunikation gegenüber der erhöhten Parallelisierung. Verstärkt wird dieser Effekt vermutlich auch dadurch, dass auf der Test-Node nur 4 der 8 Kerne Performance-Cores - und damit auf Geschwindigkeit fokussiert - sind, während die restlichen Efficiency-Cores auf Energiesparung ausgelegt sind.

3.3.2 Load-Balancing

Mit einem idealen Hardware-Setup, wie man es bei einem Cluster erwarten würde, könnte man die Berechnung der n-Body Simulation perfekt auf die verfügbaren Nodes aufteilen, sodass jeder Rank die gleiche Anzahl an Körpern berechnet, da man davon ausgehen kann, dass die Nodes über identische Rechenleistung verfügen. Im in Tabelle 1 gelisteten Setup ist dies leider nicht der Fall. Aufgrund der großen Bandbreite an Plattformen, Betriebssystemen und vor allem auch Leistungsklassen der zur Verfügung stehenden Nodes, war von Anfang an klar, dass für einen sinnvolle Verteilung der Rechenlast über mehrere Nodes hinweg Load-Balancing notwendig sein würde. Nach gründlichem Abwägen des Verhältnis aus Rechenoverhead, Genauigkeit

und Implementierungsaufwand wurde schließlich eine einfache, dynamische Load-Balancing Strategie gewählt.
Dabei wird zu Beginn eine gleichmäßige Verteilung der Körper auf die Ranks vorgenommen und in jedem Zeitschritt die Rechenzeit jedes Ranks gemessen. Nach 5 Zeitschritten wird dann die bisherige durchschnittliche Rechenzeit pro Zeitschritt für jeden Rank berechnet und die Körper so umverteilt, dass die Rechenzeit der Ranks sich annähert. Die restlichen Zeitschritte werden dann mit dieser neuen Verteilung berechnet. Es wird also nur einmal pro Simulation umverteilt, was den Overhead der Umverteilung auf ein Minimum reduziert und damit Performance Messungen kaum beeinflusst, ohne die Vorteile von MPI durch eine schlechte Verteilung der Rechenlast zu verschenken.

3.3.3 MPI in Windows

Die naheliegendste Gerätewahl für einen ersten Versuch mit MPI fiel aufgrund der ohnehin bereits eingerichteten Entwicklungsumgebungen auf die beiden Windows-Geräte, die auch zu den leistungsfähigsten der zur Verfügung stehenden Nodes zählten. Leider stellte sich jedoch die Implementierung von MPI in einer reinen Windows-Umgebung als schwieriger heraus als erwartet. Nach anfänglichen Problemen mit der Benutzerverwaltung von Windows, dem alles andere als trivialen Ausführen von Programmen als ein anderer Benutzer sowie den Firewall-Einstellungen, die die Kommunikation zwischen den Ranks verhinderten, scheiterte die Umsetzung auf Windows schließlich. Vermutlich verbaten bis zuletzt Firewall Regeln den Aufbau der Verbindung. Genau konnte dies nicht mehr nachvollzogen werden, da die Fehlersuche in diesem Bereich sehr zeitaufwendig war.

3.3.4 MPI in Docker

Da die Workstation und der Mac die mit Abstand Leistungsfähigsten der zur Verfügung stehenden Systeme waren, war von Anfang an ein ideales Ziel, beide mit MPI zu verbinden. Als dann auch klar wurde, dass die Umsetzung in einer reinen Windows-Umgebung schwieriger als gedacht war, rückte Docker mehr in den Fokus.

Docker erlaubt das Erstellen und Ausführen von Containern - Linux-basierte Images eines Operating Systems mitsamt installierter Programme und Dateien - in speziellen, Hardware-nahen Virtual Machines mit minimalem Overhead.
Docker läuft am Besten auf Linux Betriebssystemen, unterstützt aber mit *Docker Desktop* auch Windows und macOS. Durch die Linux Kompatibilität bereits gut geeignet für drei der verfügbaren Nodes, bestand die Hoffnung mithilfe von Docker alle Nodes ungeachtet des Betriebssystems durch eine simple Konfiguration zusammenzuschalten.

Doch die Eigenheiten des Container Systems von Docker verursachten Schwierigkeiten in der Kommunikation zwischen den Ranks. Während das gemeinsame Starten der Simulationsberechnung durch SSH gelang, konnten die Ranks keine TCP-Verbindung zueinander aufbauen.

Abhilfe schaffte hier *Docker Swarm*. Swarm ist ein Docker Feature zur Erzeugung und Verwaltung von Containern über mehrere Maschinen hinweg. Dabei wird ein virtuelles Netzwerk von der Master-Maschine erstellt, dem die Arbeiter dann beitreten. Mit Swarm gelang die Kommunikation zumindest zwischen den Linux-Nodes.
Die macOS Node konnte sich erst verbinden, als Docker und der Container innerhalb einer

Linux-VM ausgeführt wurden. Auch wenn diese doppelte Virtualisierung gegenüber einer nativen Ausführung langsamer ist, stand jetzt eine flexible und leicht einzurichtende MPI Konfiguration zur Verfügung, die wie in Abschnitt 3.3.5 beschrieben deutlich messbare Performance-Verbesserungen ermöglichte.

3.3.5 Auswertung

Im direkten Vergleich mit „reinem“ OpenMP auf einer Maschine (vgl. Abbildung 15, 16) wurden sowohl die Schwächen als auch die Stärken von MPI sichtbar - während eine kleine Union aus den zwei schnellsten Nodes (Workstation und Mac) die einzelnen Maschinen übertrumpfte, hinkte die Laufzeit der Zusammenschaltung von Workstation, ThinkPad, Mac und HP deutlich hinter reinem OpenMP her.

Auch diese Unterschiede lassen sich erneut auf Netzwerk-Overhead zurückführen, sowie Unterschiede in der Leistungsfähigkeit.
MPI ist dann effizient, wenn große Teile der Rechenleistung auf mehrere Nodes verteilt werden können, und dabei die Rechenzeit über die Netzwerkübertragungs-Zeit signifikant überwiegt. Im Fall der Zusammenschaltung aller vier verfügbaren Nodes waren zwei allerdings deutlich langsamer als die anderen, konnten also nur einen kleinen Teil der Bodies übernehmen. Trotzdem wurde ein großer Teil der Rechenzeit mit dem Übertragen und Synchronisieren dieser zwei Nodes verschwendet.
Mit höheren Körper-Anzahlen kann dieser Overhead verringert werden, da die Rechenzeit mit mehr Körpern schneller ansteigt als die Übertragungszeit, trotzdem bleibt diese große Zusammenschaltung eher ineffizient.

Anderes gilt für die Kombination von Mac und Workstation. Beide verfügen über ähnliche Rechenleistung, können also die Körper perfekt unter sich aufteilen. Auch müssen sie sich nur mit jeweils einer anderen Node synchronisieren, was den Netzwerk-Overhead reduziert.
Auch hier gilt, dass die Effizienz mit mehr Körpern steigt. Das zeigte sich auch in der CPU auslastung des Mac, der als Worker-Node bei ca. 60% begann und sich mit höheren Körper-Anzahlen den 100% annäherte.

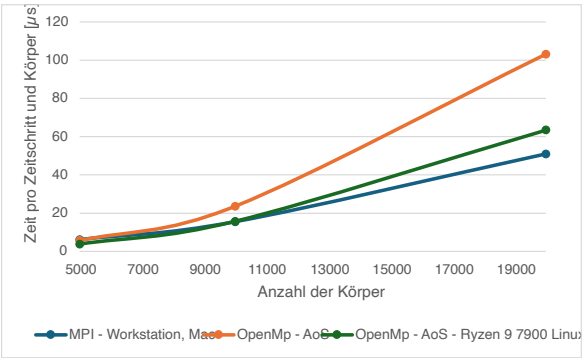


Abbildung 15: MPI Vergleich

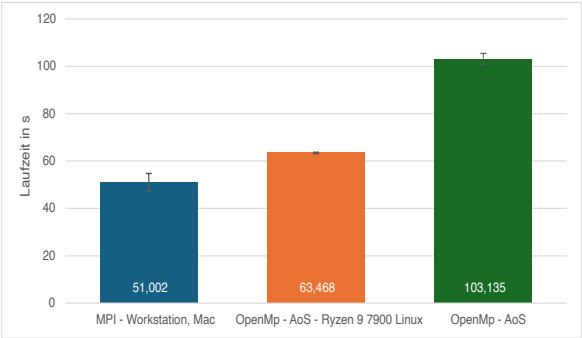


Abbildung 16: MPI Vergleich - 20k

3.4 OpenCL Parallelisierung

In den vorangegangenen Abschnitten konnte gezeigt werden, dass n-Body-Simulations mit entsprechenden Optimierungen bereits einigermaßen effizient auf einer CPU berechnet werden können.
Trotzdem ist die CPU eigentlich nicht das beste Werkzeug für diese Art von Berechnung. Jeder Zeitschritt der Simulation ist das Ergebnis von tausenden Daten-Parallelen, und mehr rechen- als speicherintensiven Berechnungen und daher perfekt geeignet für die Berechnung auf einer GPU.

Für Berechnungen auf der GPU hat man im allgemeinen die Wahl zwischen OpenCL, das auf allen gängigen Plattformen verfügbar ist, und CUDA, das speziell für NVIDIA GPUs geschrieben ist.
Um Plattformunabhängig zu sein, und da die Haupt-Entwicklungsgeräte keine NVIDIA GPU hatten, fiel die Wahl auf OpenCL obwohl mit der GPU der Workstation, einer RTX 3090 grundsätzlich Zugang zu einer CUDA-kompatiblen GPU bestand.

Auffällig an den OpenCL-Implementierungen ist, dass auf der Haupt-Testmaschine, dem MacBook, die GPU zwar jede CPU-Basierte Implementierung um ein Vielfaches schlug, zwischen den einzelnen OpenCL-Implementierungen aber - die auf der NVIDIA GPU deutliche Verbesserungen brachten - kaum ein Unterschied messbar war.
Im Nachfolgenden wurden daher in reinen GPU-Performance Vergleichen vor allem Messwerte der NVIDIA RTX 3090 untersucht.

3.4.1 OpenCL-Umsetzung bisheriger Optimierungen

Über den Entwicklungszeitraum wurden analog zu OpenMP Implementierungen in mehreren Iterationen erstellt.
Ausgangspunkt war eine klassische AoS Implementierung, bei der die Berechnung der Kraft auf einen Körper jetzt nicht mehr auf CPU-Threads aufgeteilt wurde, sondern für jeden Körper ein GPU-Thread erstellt wurde. Bereits diese Implementierung war aufgrund der hervorragenden Parallelisierung der GPU schneller als jede CPU-basierte Variante.

Anschließend wurden auch nacheinander SoA Varianten analog zu OpenMP implementiert, und mit der Split-Loop Optimierung verbessert.
Ähnlich wie die SIMD-Vektorisierung der CPU profitiert die GPU dank dem SIMT Konzept, also der Ausführung derselben Instruktion auf mehreren gruppierten Threads, stark von der Vermeidung von Verzweigungen im Kernel und Zusammenhängenden Daten im Speicher (vgl. Abbildung 18). Insbesondere Speicherzugriffe werden schnell zum Engpass in der GPU-Berechnung.

3.4.2 Shared Memory

Genau da setzt die Speicheroptimierung an. GPU-Speicher sind in mehrere Level aufgeteilt.
Ganz außen steht das Global Memory, das von allen Threads in der GPU verwendet werden kann. Darauf folgt auf Block-Ebene das Shared Memory, das nur noch von Threads in einem Block verwendet werden kann, aber schneller ist. Auf unterster Ebene liegt das schnellste und Kleinste Local Memory, das nur noch ein einzelner Thread verwenden kann.

Die Idee war, anstatt jeden Thread die benötigten Daten für sich aus dem Speicher laden zu lassen, Threads in Blocks aufzuteilen, die sich die Daten über Shared Memory teilen.
So kann jeder Thread nur einen Einzelnen Wert aus dem Globalen Speicher laden, über eine Barrier warten, bis die Nachbarn aus dem Block ihre eigenen Werte geladen haben und dann einen Teil seiner Workload berechnen. [7]
Natürlich kann so nur ein kleiner Teil der Daten zur gleichen Zeit im Shared Memory existieren, also müssen Threads nach der Ausführung wieder warten bis ihre Nachbarn ebenfalls fertig sind, um einen neuen Satz an Daten zu laden.

Trotzdem spart ein Thread sich je nach Blockgröße - typischerweise etwa 128 oder 256 - so viele Global Memory Zugriffe wie er Nachbarn hat. Dadurch konnte eine weitere Verbesserung der Performance erreicht werden (vgl. Abbildung 18).

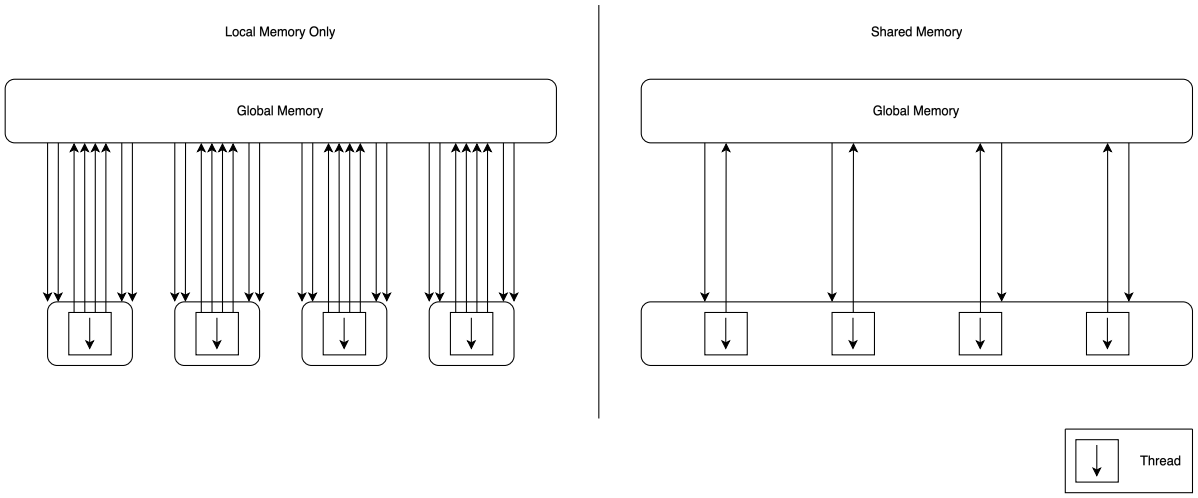


Abbildung 17: Nur Local Memory vs. Shared Memory

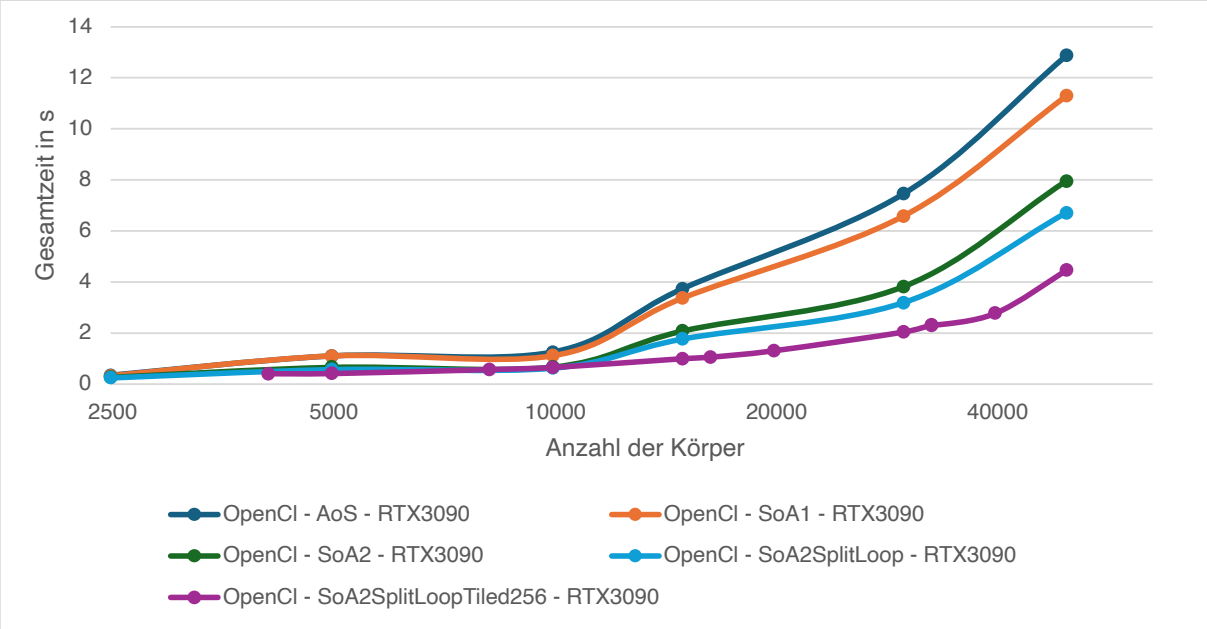


Abbildung 18: OpenCL Performance Vergleiche

3.5 Abweichungen im Ergebnis zwischen Algorithmen

Ein wichtiger Aspekt in der Optimierung von Algorithmen ist, dass die Ergebnisse über die Iterationen (nahezu) konstant bleiben - ein Programm kann noch so schnell sein, wenn es falsche Ergebnisse liefert ist es unbrauchbar.

Über die Entwicklung wurden daher sowohl visuelle als auch mathematische Vergleiche der Simulationsergebnisse durchgeführt.

Visuell produzierten alle Algorithmen ähnlich aussehende Ergebnisse - durch die Anzahl an Körpern sind für das Auge hauptsächlich die entstehenden Muster erkennbar. Im mathematischen Vergleich der Endergebnisse offenbarten sich jedoch teilweise massive Unterschiede zwischen den Implementierungen.

So kamen die OpenCL Implementierungen nie auf die gleichen Ergebnisse wie ihre CPU-basierten Gegensätze, und auch die verbesserte SoA Version produzierte sowohl auf CPU als auch auf GPU andere Ergebnisse als die AoS Variante, obwohl in der Theorie dieselben Rechnungen durchgeführt werden.

Diese Abweichungen lassen sich hauptsächlich auf Unterschiede in der Fließkommazahl-Berechnung zurückführen. Fließkommazahlen sind auf Computern von Natur aus ungenau und können nur angenähert richtig berechnet werden. Die Ungenauigkeiten einzelner Berechnungen sind sehr klein und fallen in den meisten Fällen nicht auf, bei n-Body-Simulations verstärken sie sich allerdings über die vielen Zeitschritte sehr schnell und erzeugen eine hohe Divergenz. Bei wenigen Timesteps und Bodies waren die Ungenauigkeiten kaum messbar, bei höheren Anzahlen führten sie allerdings zu völlig unterschiedlichen Ergebnissen.

Diese Ungenauigkeiten lassen sich auf Kosten von Geschwindigkeit durch Deaktivierung von *fast-math* Compiler Flags und Angleichung des Codes etwas verringern, allerdings Hardware-bedingt nie vollständig eliminieren.

Für dieses Projekt ist das jedoch auch nicht problematisch, da hier keine exakten physikalischen Werte berechnet werden müssen sondern der Fokus auf der Optimierung der Geschwindigkeit liegt. Durch die diskrete, Zeitschritt-basierte Berechnung ist diese n-Body-Simulation von Natur aus nur eine Näherung der real wirkenden Kräfte. Fließkommazahl-Abweichungen sind also nur ein Teil der Ungenauigkeiten dieser Simulation.

3.6 Zusammenfassung

Die Ergebnisse der Implementierung und Messung bestätigen die Vermutungen der initialen Analyse.

Die äußere Schleife einer n-Body-Simulation lässt sich hervorragend parallel ausführen und die besten Ergebnisse wurden auf einer GPU erzielt. Nach allen Optimierungen lag die höchste Anzahl Körper, die noch in einer verhältnismäßig effizienten Zeit berechnet werden konnte bei ca. 2 Millionen, für die 750 Zeitschritte innerhalb von etwa einer Stunde berechnet wurden (vgl. Abbildung 19).

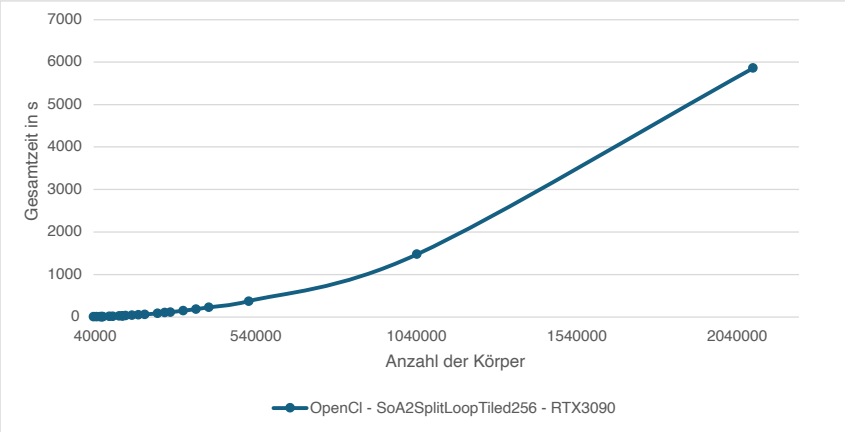


Abbildung 19: OpenCL Laufzeit für sehr große n

Trotzdem lassen sich durch Multithreading, kluges Speicherlayout und Optimierung für Vektorisierung auch gute Geschwindigkeitswerte auf der CPU erzielen. MPI bleibt aufgrund fehlender SoA-Split-Loop Optimierung knapp hinter OpenMP, kann aber auch mit entsprechender Hardware und schneller Netzwerkverbindung die Grenzen einer einzelnen CPU signifikant erweitern.

Insgesamt zeigen die Ergebnisse in Abbildung 20, dass erst die Optimierungen und Verwendung paralleler Berechnungen eine sinnvolle Simulation des n-Body-Problems ermöglichen.

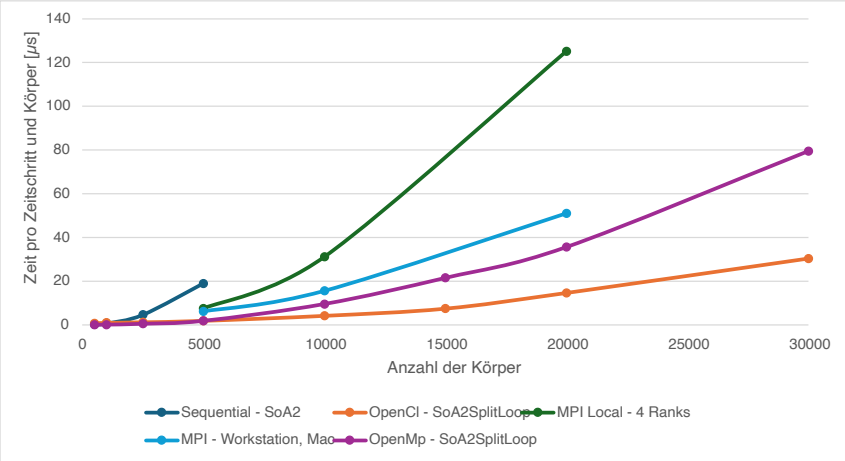


Abbildung 20: Vergleich der verschiedenen optimierten Verfahren

4 Ausblick

4.1 MPI und OpenCL

Im Abschnitt über MPI wurde bewiesen, dass MPI die Laufzeiten mit entsprechend Leistungsfähiger Hardware signifikant senken kann.
Allerdings wurde zur tatsächlichen Berechnung innerhalb des Ranks mit dem OpenMP AoS Algorithmus eine vergleichsweise langsame Implementierung getestet.

Interessant wäre neben der Verwendung schnellerer OpenMP Algorithmen hier das Zusammenspiel mit OpenCL, wenn mehrere Nodes mit starker Grafikkarte zusammengeschaltet werden. Dadurch könnten eventuell sogar die aktullen Spitzenwerte der reinen OpenCL Variante unterboten werden.

Herausforderung dabei wäre vor allem die Synchronisation. Erstens würden aufgrund der höheren Körper-Anzahlen deutlich größere Datenmengen synchronisiert werden müssen (auch wenn Körper grundsätzlich sehr kleine Objekte sind). Zweitens profitiert der aktuelle OpenCL Algorithmus sehr davon, dass die Daten zwischen einzelnen Zeitschritten im GPU-Speicher bleiben können und nicht neu vom RAM kopiert werden müssen. Wenn diese Daten also über das Netzwerk synchronisiert - und dabei herunter- und wieder hochgeladen - werden müssen, könnte dieser Overhead zu intensiv werden.
Um effektiv auf mehrere Grafikkarten zu skalieren wäre also unter Umständen eine Direkt-Verbindung mehrerer Grafikkarten über bspw. PCIe besser geeignet.

4.2 Barnes-Hut Algorithmus

Eine weitere algorithmische Optimierung, die in diesem Projekt nicht implementiert wurde, aber für die Berechnung von sehr großen n-Body-Simulationen äußerst vorteilhaft ist, ist der Barnes-Hut Algorithmus. Dieser Algorithmus reduziert die Zeitkomplexität von $O(n^2)$ auf $O(n \log n)$, indem er die Kräfte zwischen weit entfernten Körpern approximiert, anstatt sie genau zu berechnen.
Der Barnes-Hut Algorithmus teilt den Raum in eine hierarchische Struktur von Quadranten (in 2D) oder Oktanten (in 3D) auf, die als Baum dargestellt wird. Jeder Knoten des Baums repräsentiert eine Gruppe von Körpern, und die Kräfte zwischen einem Körper und einer Gruppe von Körpern werden approximiert, wenn die Gruppe weit genug entfernt ist.
Die Implementierung des Barnes-Hut Algorithmus würde es ermöglichen, deutlich größere n-Body-Simulationen effizient zu berechnen, da die Anzahl der Berechnungen erheblich reduziert wird. Allerdings ist die Implementierung komplexer als die direkte Berechnung und erfordert sorgfältige Handhabung der Datenstrukturen und der Approximation, um genaue Ergebnisse zu gewährleisten.
Nimmt man jedoch diese Hürden erfolgreich, bieten sich ganz neue Möglichkeiten der Parallelisierung und Optimierung, da nun nicht mehr alle Körper miteinander interagieren müssen, sondern nur noch die Gruppen von Körpern. Dadurch wird die Menge an Daten, die zwischen den Threads oder Nodes synchronisiert werden muss, erheblich reduziert. Ein Grafikkarten MPI-Cluster könnte so vermutlich Milliarden von Körpern berechnen, was mit der direkten Berechnung selbst mit mehr Zeitaufwand unmöglich wäre.

Literaturverzeichnis

[1] I. Advanced Micro Devices, *AMD EPYC^{TM} 9965*. Adresse: https://www.amd.com/en/products/processors/server/epyc/9005-series/amd-epyc-9965.html.

[2] S. Gerstl, "Kilocore: Weltweit erster Mikrochip mit 1.000 Kernen", *Storage Insider*, 2016. Adresse: https://www.storage-insider.de/kilocore-weltweit-erster-mikrochip-mit-1000-kernen-a-541425/.

[3] C. Murray und S. Dermott, *Solar System Dynamics*. Cambridge University Press, 1999.

[4] W. Press, S. Teukolsky, W. Vetterling und B. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3. Aufl. Cambridge University Press, 2007.

[5] RPTU, *Guidelines for Vectorization*. Adresse: https://hpc.rz.rptu.de/Tutorials/AVX/guideline.shtml.

[6] P. Tipler und G. Mosca, *Physik für Wissenschaftler und Ingenieure*, 8. Aufl. Springer Spektrum, 2019.

[7] A. Tortolero, *Threads in Harmony*. Adresse: https://www.thecolorcode.net/post/24.

Abbildungsverzeichnis

1 n-Body-Simulation Paralellisierbarkeit . . . 3
2 Geschwindigkeit Sequenziell vs. OpenMP . . . 5
3 Bodies als AoS . . . 5
4 Bodies als SoA . . . 5
5 Bodies als verbesserte SoA . . . 6
6 Sequenziell AoS vs SoA . . . 6
7 If-Statement innere Schleife . . . 7
8 Innere Schleife Split-Loop Lösung . . . 7
9 OMP Vergleich . . . 7
10 OMP Vergleich - 30k . . . 7
11 OMP Single-Loop im Vergleich zu OMP AoS . . . 8
12 Single-Loop Version auf dem MacBook Air (2500 Körper) . . . 9
13 Single-Loop Version auf der Workstation (2500 Körper) . . . 9
14 OpenMP AoS vs. MPI auf einer Node (5k Körper) . . . 10
15 MPI Vergleich . . . 12
16 MPI Vergleich - 20k . . . 12
17 Nur Local Memory vs. Shared Memory . . . 14
18 OpenCL Performance Vergleiche . . . 14
19 OpenCL Laufzeit für sehr große n . . . 16
20 Vergleich der verschiedenen optimierten Verfahren . . . 16

Tabellenverzeichnis

1 Verfügbare Nodes 9